

Automation Builder 建工程操作卡(零基础·中英对照版 v2.1)

你在干什么:把写好的 PLC 程序(plc\ 里的 .st 文本文件)装进 Automation Builder(AB), 在仿真模式 (电脑里虚拟一台 PLC, 不用真机) 下跑起来。当前已验收证据是 59/0; 工作区当前显示 75 项, 其中 T60-T69 是真实断言、T70-T75 是恒 TRUE 预留, 且新增 CB/TOB/lex 源码尚未 AB 复测, 不能把 75/0 当已验证结论。

时长:第一次 90-120 分钟。**门槛:**会复制粘贴即可,不需要懂编程。

AB 界面基本是英文的——本卡所有操作词都按「English(中文)」写,照着英文找按钮。

总原则:每步都有"预期看到什么";不符就整窗截图发 Claude,判断问题是我的活不是你的。

第 0 步:认识界面(5 分钟,只看不点)

启动 AB,记住三个区域:

- 1. **左侧竖条 = 工程树,标签叫 "Devices"(设备):**目录树,我们的东西都挂在 **Application(应用)** 节点下(它藏在 PLC_AC500 → **PLC Logic** → Application 层级里,点小箭头展开)。
- 2. **中间大区域 = 编辑器(Editor):**双击树上任何东西在这里打开、粘贴代码。
- 3. **下方横条 = "Messages"(消息窗):**编译结果在这里,目标永远是 "0 errors(0 个错误)"。

术语速记(遇到回来查,不用背):

界面英文	中文	大白话
Project	工程	一个保存全部工作的"文档"
DUT (Data Unit Type)	数据类型	定义"货物"由哪些字段组成
GVL (Global Variable List)	全局变量列表	全程序共用的数据(如 400 仓位数组)
POU (Program Organization Unit)	程序单元	装代码的盒子,分 Function/Function

		Block/Program 三种
Function / Function Block / Program	函数/功能块/程序	算完即走 / 有记忆的模块 / 主入口
Build	编译	检查代码有没有错,像拼写检查
Structured Text (ST)	结构化文本	我们用的编程语言的名字

第 1 步:新建工程(10 分钟)

4. 菜单 **File** → **New Project...**(文件→新建工程)。

- 预期:弹出 New Project 对话框,左侧分类(Categories)+右侧模板(Templates)。

5. 模板选 **AC500 project**(带 PLC 图标)。下方:

- **Name(名称)**:填 `ABB_WH`
- **Location(位置)**:点 ... 浏览到 `F:\abb_wh_work\plc\`,在路径末尾补新文件夹名 `AB_Project`
- 点 **OK/Add**。

6. 接着会让你选 **PLC 型号**(标题 "New project" 的对话框:左树分类 PLC - AC500 V3,右侧型号列表):

- 右侧列表**单击选中** `PM5650-2ETH`(真实型号名带 -2ETH 后缀=双以太网口,所以搜字面

"PM5650" 找不到;它在列表靠后,描述 AC500 CPU 80MB, Ethernet)。

- 顶部 **Object name(对象名)**若空着填 `PLC_AC500`(或保留默认,不重要);
- 点 **Add PLC**(选中型号后才由灰变可点)→ 点 **Close** 关对话框。
- 预期:左侧 Devices 树出现 PLC 节点;逐层展开(PLC_AC500 → PLC Logic)能找到 **Application**。

7. 若弹许可证窗口:选 **Basic license(基础版,免费)**,按引导 **Activate(激活)**。

8. **Ctrl+S** 保存(以后每完成一步按一次)。

✓ 验收:Devices 树里能看到 Application。卡住 → 整窗截图发 Claude。

第 2 步:小试牛刀——先建 1+1 个,把流程跑通(15 分钟)

先用最小样本学会「建对象 → 粘代码 → Build」循环,成功一次,后面全是重复劳动。

2a. 第一个 DUT

9. 右键 **Application** → **Add Object(添加对象)** → **DUT....**。

- 预期:弹出对话框要求 Name(名称)和类型。

10. **Name** 填 `ST_Good`(建议从本卡复制,一个字母不能错);

类型勾 **Structure(结构)**;点 **Add(添加)**。

- 预期:编辑器打开新页,已有几行自动生成的模板(`TYPE ST_Good : STRUCT ...`
`END_TYPE`)。

11. 记事本打开 `F:\abb_wh_work\plc\01_DUTs.st`,复制从 `TYPE ST_Good :` 到第一个
`END_TYPE` 的整段。

12. 回 AB 编辑器:**Ctrl+A 全选** → **Ctrl+V 覆盖粘贴** → **Ctrl+S**。

2b. 第一个 Function(函数)

13. 右键 **Application** → **Add Object** → **POU...**。

14. 对话框:**Name** 填 `FC_CapCoef`;Type 勾 **Function(功能/函数)**;

出现 **Return type(返回类型)**框 → 填 `REAL`;

Implementation language(实现语言)选 **Structured Text (ST)**。点 **Add**。

- 预期:编辑器页面**分上下两窗**——上窗 = Declaration(声明区),下窗 = Implementation/Body(实现区)。

15. 打开 `plc\03_Functions.st` 找 `FC_CapCoef` 段:

- `FUNCTION FC_CapCoef : REAL` 到最后一个 `END_VAR` → 粘**上窗**(先 **Ctrl+A** 清模板);
- `END_VAR` 之后到 `END_FUNCTION` 之前的逻辑 → 粘**下窗**;

- 若你的版本只有单窗:整段(含首尾行)一起粘。

16. **Build 一次**:菜单 **Build** → **Build**,或按 **F11**。

- 预期:Messages 窗显示 **0 errors**(黄色 warnings 不用管)。

✓ 验收:0 errors——流程通了。✗ 有红色 error:双击那行看跳哪,截图(含英文报错)发 Claude。

第 3 步:补齐数据类型与全局变量(20 分钟)

3a. 再建 4 个 DUT(方法同 2a):

Name	类型勾选	代码来源
ST_Slot	Structure(结构)	01_DUTs.st
ST_Stats	Structure	01_DUTs.st
E_MainState	Enumeration(枚举)	01_DUTs.st
ST_TestVec	Structure	07_GVL_Data_generated.st 开头

3b. 建 4 个 GVL:右键 Application → Add Object → **Global Variable List...**(全局变量列表):

Name	代码来源	特别说明
GVL_Param	02_GVL.st	该段有两个 VAR_GLOBAL 块 (CONSTANT 和普通),都粘进这一个
GVL_WH	02_GVL.st	—
GVL_Visu	02_GVL.st	—
GVL_Data	07_GVL_Data_generated.st	也是两个块都粘

规则同前:打开后 Ctrl+A 清模板再粘。每建一个按一次 F11,保持"随时 0 errors"——

一报错就知道是刚才那一步,好定位。

第 4 步:导入全部函数(20 分钟)

重复 2b 的方法(POU → Function → 填 Return type → 上下窗粘贴),共 13 个:

Name	Return type	来源
FC_CapCoef	REAL	第 2 步已完成 ✓
FC_IsPresetOccupied	BOOL	03_Functions.st
FC_CheckFeasible	BOOL	03
FC_FitsPhysical	BOOL	03
FC_CalcTravelTime	REAL	03
FC_CalcPathLen	REAL	03
FC_TMax	REAL	03
FC_CalcScore	REAL	03
FC_CalcPriority	REAL	03
FC_AxisTime	REAL	03
FC_CalcDualCycleTime	REAL	03
FC_StateToColor	DWORD	03(T17 可视化配色查表)
FC_LoadDemoGoods	BOOL	07 文件末尾

技巧:源文件里 Ctrl+F 搜函数名定位;每段边界=FUNCTION 名字 : 类型 到 END_FUNCTION。

每建 2-3 个按一次 F11。

第 5 步:功能块与主程序(20 分钟)

5a. 9 个 Function Block(功能块):Add Object → POU → Type 勾 **Function Block**

(这种没有 Return type 框),来源全部 04_FB_Warehouse.st

(段落边界 FUNCTION_BLOCK 名字 → END_FUNCTION_BLOCK):

FB_InitWarehouse、FB_SelectSlot、FB_AssignAllGoods、FB_LocalSwapImprove、
FB_Stats、FB_AnimatePath、FB_ScanLoadProbe、FB_BuildVisuPath、
FB_VisuRefresh(T17 可视化镜像)。

5b. 2 个 Program(程序):Add Object → POU → Type 勾 **Program**:

PRG_Main(来源 05_PRG_Main.st)、PRG_Test(来源 06_PRG_Test.st)。

全部粘完 F11:0 errors 才继续。(plc 里的 08 号文件暂不用导,后续接线用。)

第 6 步:把程序挂上 Task(10 分钟)

概念:PLC 是"每 10 毫秒把程序从头到尾跑一遍"的循环机器;Task(任务)就是那个闹钟。

17. Devices 树找 **Task Configuration(任务配置)**(在 Application 下),展开——

通常已有一个现成任务(名叫 Task 或 MainTask)。双击它。

18. 编辑器里:Type(类型)应为 **Cyclic(循环)**;Interval(间隔/周期)改为

t#10ms(显示 20ms 就改 10ms)。

19. 同页找 **Add Call...(添加调用/添加 POU)**按钮 → 列表里选 PRG_Main → OK;

再点一次 Add Call → 选 PRG_Test → OK。

- 预期:任务下方列表出现这两个程序名。

20. F11(0 errors),Ctrl+S。

第 7 步:仿真运行(10 分钟)

三个概念:Simulation(仿真)=电脑里虚拟一台 PLC;Login(登录)=把程序装进去;

Start(启动)=按下它的运行键。

21. 菜单 **Online** → **Simulation(在线→仿真)**,点一下打上勾。

- 预期:窗口底部出现红色 **SIMULATION** 字样(红色只是提醒,不是错误)。

22. 菜单 **Online** → **Login(登录)**,或 **Alt+F8**。弹确认框(一般问

"...download...?" 要不要下载)→ 点 **Yes**。

- 预期:Application 节点变绿/显示 [run] 或 [stop]。

23. 菜单 **Debug** → **Start(调试→启动)**,或 **F5**。

- 预期:状态变 **RUN**(绿色)。

24. 双击 PRG_Main:现在是**在线模式(online view)**,每个变量后面实时显示当前值。

- 预期:eState 显示 **STATE_WAIT_INPUT**(程序已初始化完仓库,在等指令)。

第 8 步: 复核已验收的 59 项基线 (15 分钟)

在线改变量值的标准手法(记住,演示也用它):

找到变量那一行的 "**Prepared value**"(准备值)列 → **双击**,出现 TRUE

(或弹小窗让你填,填 TRUE)→ 菜单 **Debug** → **Write values(写入值)**,快捷键 **Ctrl+F7**。

25. 在线状态下双击打开 **PRG_Test**。

26. 变量 **xRunTests** 按上述手法写 **TRUE**。

- 程序瞬间跑完并自动复位它,看不到它变 TRUE 是正常的。

27. 冻结的已验收基线在干净初态+当前参数下应为: **`iPassed`=59、`iFailed`=0、**

`xAllPass`=TRUE。当前源码虽有 T60-T69 真实断言, T70-T75 仍是预留;只有在全部待测项明确、完成新一轮 AB 复测并保存读回依据后,证据才能升级。否则仍只承认 59/0。上述结果不外推为 H KPI、权限安全或全部专项。

- 其中静态匀速黄金向量只形成有限一致性护栏;通过不等于 H、自适应或 lex/speed 在 PLC 复现。

28. **截图**(画面里要有 iPassed/iFailed),存 **F:\abb_wh_work\plc\验收截图**

(没有就新建),命名 **w1_PRGTest_59pass.png** (若未来真实扩容,再按实测数另存新文件,不覆盖历史证据)。

29. 若 iFailed ≠ 0:看 **sLastFail** 显示的编号(如 T13),截图发 Claude。

第 9 步:顺手看一眼演示(10 分钟,推荐)

30. 在线状态双击 **GVL_Visu**,依次写值(手法同第 8 步):

CmdReset=TRUE(清测试数据重新初始化)→ **CmdLoadDemo**=TRUE(载入 20 件演示货,
与 Python 同一批)→ **SelStrategy**=**3**(选 AWRA-LS)→ **CmdRunAssign**=TRUE(开始!).
31. 盯 **VisuStatusText**:依次显示 **ASSIGN_RUN_PLACED=...**(分配中)→ **IMPROVE_R=..._SWAPS=...**
(优化中)→ **‘DONE_OK’**(完成,全部入库违规=0)。

(状态文本用英文码是因为 PLC 字符串里的中文会引发编码警告 C0555;决赛画面上会配中文
标签。)

32. **GVL_WH** 里 **stStats**:PlacedCnt=20、ViolCnt=0。截图留档。

这就是决赛可视化的数据底座,W4 给它配 20×20 图形界面和动画。

中英对照速查表(找不到东西先查这里)

你要找的	界面英文	在哪
新建工程	File → New Project	顶部菜单
工程树	Devices	左侧面板标题
应用节点	Application	PLC_AC500 → PLC Logic 下
添加对象	Add Object	右键 Application
数据类型/结构/枚举	DUT / Structure / Enumeration	Add Object 对话框
全局变量列表	Global Variable List (GVL)	Add Object 对话框
程序单元/函数/功能块/程序	POU / Function / Function Block / Program	Add Object 对话框
返回类型 / 实现语言	Return type / Implementation language	建 POU 对话框
结构化文本	Structured Text (ST)	语言下拉框
声明区 / 实现区	Declaration / Implementation(或 Body)	POU 编辑器上/下窗
编译	Build → Build(F11)	顶部菜单

消息窗 / 0 错误	Messages / 0 errors, 0 warnings	底部面板
任务配置 / 循环 / 周期	Task Configuration / Cyclic / Interval	Application 下
添加程序调用	Add Call(或 Add POU)	任务编辑页按钮
仿真	Online → Simulation	顶部菜单(勾选)
登录 / 下载	Login(Alt+F8)/ Download	Online 菜单 / 弹窗
启动 / 停止	Debug → Start(F5)/ Stop(Shift+F8)	顶部菜单
运行状态	RUN / STOP	状态栏
准备值 / 写入值	Prepared value / Write values(Ctrl+F7)	在线变量表列 / Debug 菜单
许可证	License / Activate Basic license	首次启动引导

常见报错速查(Messages 窗红字)

英文报错关键词	意思/原因	处理
Unknown type: 'ST_Good'	数据类型没建或名字错	核对第 2/3 步 5 个 DUT
Identifier 'GVL_Param' not declared/found	全局变量表没建全	核对第 3b 步 4 个 GVL
Identifier 'FC_xxx' not found	某函数漏建	对照第 4 步表逐个数
'MAX' ... unexpected	标准库没挂上	截图发 Claude,给替换补丁
CONCAT ... too many parameters	老版本字符串函数限两参	截图发 Claude,给拆分补丁
'STATE_IDLE' is no component of ...	枚举写法版本差异	全文删 E_MainState. 前缀再编译,并告诉 Claude
Cannot connect / No device	忘了勾 Simulation	回第 7 步第 1 条
中文注释乱码	编码差异	无害,不影响运行

万能兜底:任何一步与预期不符 → 整窗截图 + 说明做到第几步 → 发 Claude。

全部通过后回我一句 “iPassed=59,iFailed=0” （若不是 59 说明工程版本不一致，先停下核对）。这一遍下来你已掌握 AB 的工程结构、四类对象、

Build/Simulation/Login/写值——决赛可视化(W4)要用的全部基本功。